Appendix

A Neptune Inputs

Neptune takes as input a compute definition of the operator to optimize, and a schedule that describes how to optimize it.

Figure 12 shows one possible Neptune input, a flexible compute definition for attention. It allows customizing bias and mask conditions, and in 38 lines of code it covers all operators in Table 1 except GQA operators.

Figure 13 shows the schedule that pairs with the compute definition. In 28 lines of code, it applies rolling update to fuse the attention computation into a single loop nest, and produces many of the high-performance kernels in our evaluation.

```
def create_general_attention(
    B: int, N: int, QS: int, KVS: int, H: int, mask_cond: Callable, score_mod: Callable
):
  q = placeholder((B, N, QS, H), "float16", name="q")
  k = placeholder((B, N, KVS, H), "float16", name="k")
  v = placeholder((B, N, KVS, H), "float16", name="v")
  p = batch_matmul(q, k, trans_b=True, out_dtype="float32")
  score = compute(
      p.shape, lambda *ax: if_then_else(mask_cond(*ax), score_mod(p(*ax), *ax), float("-inf")),
      name="score_mod")
  j = reduce_axis((0, KVS), name="j")
  s_max = compute(
      (B, N, QS), lambda b, n, i: max(score(b, n, i, j), axis=j), name="softmax_maxelem")
  s_exp = compute(
      (B, N, QS, KVS), lambda b, n, i, j: exp(score(b, n, i, j) - s_max(b, n, i)),
      name="softmax_exp")
  s_expsum = compute(
      (B, N, QS), lambda b, n, i: sum(s_exp(b, n, i, j), axis=j), name="softmax_expsum")
  s_exp = compute(
      p.shape, lambda *axes: s_exp(*axes).astype(q.dtype), name="softmax_exp_f16")
  sv = batch_matmul(s_exp, v, trans_b=False, out_dtype="float32")
  sv = compute(
      sv.shape, lambda b, n, i, j: sv(b, n, i, j) / s_expsum(b, n, i), name="softmax_norm")
  return compute(sv.shape, lambda *i: sv(*i).astype(q.dtype), "cast")
```

Fig. 12. Compute definition for the attention operator, which Neptune takes as input. This definition covers all operators in Table 1 except GQA operators.

```
def schedule_attn_with_rolling_update(sch: Schedule):
  b0 = sch.get_block("batch_matmul_1")
  b1 = sch.get_block("T_score_mod")
  b2 = sch.get_block("T_softmax_maxelem")
  b3 = sch.get_block("T_softmax_exp")
  b4 = sch.get_block("T_softmax_exp_cast")
  b5 = sch.get_block("T_batch_matmul_NN")
  b6 = sch.get_block("T_softmax_expsum")
  b7 = sch.get_block("T_softmax_norm")
  b8 = sch.get_block("T_cast")
  *axes, i, j, k = sch.get_loops(b0)
  i0, j0 = sch.tile([i, j], [128, 32])
  sch.compute_at(sch.cache_read(b0, 0, "shared"), i0)
  sch.bind_block_idx([*axes, i0], ["blockIdx.x", "blockIdx.y", "blockIdx.z"])
  sch.reverse_compute_at(b1, j0)
  b2rf = sch.rolling_update(b2, j0, factor_axis=0)
  b6rf = sch.rolling_update(b6, j0, factor_axis=0)
  b5rf = sch.rolling_update(b5, j0, factor_axis=0)
  sch.reverse_compute_at(b7, i0)
  sch.reverse_compute_at(b8, i0)
  for blk in [b0, b1, b2, b2rf, b3, b4, b5, b5rf, b6, b6rf, b7]:
      sch.set_scope(blk, 0, "shared")
  sch.split_scan_buffer(b2, j0, 0)
  sch.decompose_reduction(b5, j0)
  sch.decompose_reduction(b6, j0)
```

Fig. 13. Schedule for the attention operator, which Neptune takes as input. This schedule pairs with the compute definition in Figure 12.

B Step-By-Step Algorithm Application on Motivational Example

Here we include the detailed steps of our rolling update algorithm for the example from Figure 1a.

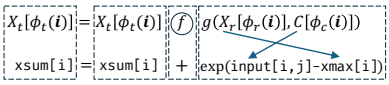
B.1 Rolling Update

```
# s_max:
for i in range(2):
  for j in range(4):  # loop_j
    xmax[i] = max(
      xmax[i], inp[i, j])
# s_sum:
for i, j in grid(2, 4)
  xsum[i] = xsum[i] + exp(
    inp[i, j] - xmax[i])
```

(a) Step 1: dataflow reorganization. This step inlines s_exp (into s_sum) and finds $\mathcal{L}_r = \{\texttt{s_max}\}$.

```
for i in range(2):
  for j in range(4):  # loop_j
    # s_max:
    xmax[i] = max(
      xmax[i], inp[i, j])
    # s_sum:
    xsum[i] = xsum[i] + exp(
      inp[i, j] - xmax[i])
```

(b) Step 2: loop transformation. Naive loop fusion fuses s_sum and s_max under loop_j.



$X_t = \texttt{xsum}$; $X_r = \texttt{xmax}$; $C = \texttt{inp}$

$f = +$; $g(x, c) = \exp(c - x)$

(c) Step 3: pattern matching on the compute statement of $L_t = \texttt{s_sum}$.

$$y = g(x, c) = \exp(c - x) \;\Rightarrow\; c = g_c^{-1}(x, y) = x + \ln(y)$$

$$h(t, r, r') = g(r', r + \ln(t)) = \exp(r - r') \cdot t$$

Proving commutativity with f:

$$h(x+y, r, r') = \exp(r-r')\cdot(x+y) = h(x, r, r') + h(y, r, r')$$

(d) Step 4: finding the repair function h. Find the inverse of g, use Equation 5 to find h, and prove Equation 4 (commutativity with f) for h.

For s_max, cache its output xmax for the previous iteration:

$$\texttt{xmax_0[i]} := \texttt{xmax[i]}^{\langle j-1\rangle}; \quad \texttt{xmax_1[i]} := \texttt{xmax[i]}^{\langle j\rangle}$$

Apply $h(t, r, r')$ with $r = \texttt{xmax_0[i]}$ and $r' = \texttt{xmax_1[i]}$:

$$\texttt{xsum[i]} = \exp(\texttt{xmax_0[i]} - \texttt{xmax_1[i]}) \cdot \texttt{xsum[i]} + \exp(\texttt{inp[i, j]} - \texttt{xmax_1[i]})$$

(e) Step 5: applying the repair function.

Fig. 14. All intermediate results of rolling update, when applied to Figure 1a.

B.2 Privatization

Figure 15 presents privatization combined with rolling update in Neptune. On top of the rolling update output in Figure 1c, we privatize s_max and s_sum with a split size of 2. xmax_1p and xsump are the local arrays.

```
for i in range(2):
  xmax_0[i] = -inf
  for j1 in range(2):
    for j2 in range(2):
      # max_local
      xmax_1p[i, j1] = max(xmax_1p[i, j1], inp[i, j1 * 2 + j2])
    # max_global
    xmax_1[i] = max(xmax_1[i], xmax_1p[i, j1])
    for j2 in range(2):
      # sum_local
      xsump[i, j1] += exp(inp[i, j1 * 2 + j2] - xmax_1[i])
    xsum[i] = exp(xmax_0[i] - xmax_1[i]) * xsum[i] + xsump[i, j1]
    xmax_0[i] = xmax_1[i]
```

Fig. 15. Privatization combined with rolling update in Neptune.

C Additional Example Operators Amenable to Neptune Transformations

Attention definition. The Attention operator is a sequence of four compute steps: matrix multiplication (matmul), element-wise score computation (division), softmax, and another matmul. Attention typically use a numerically stable softmax implementation, subtracting the input by per-row maximum to prevent exponential overflow. Equation 12 shows the compute steps of Attention over 2D inputs $q, k, v \in \mathbb{R}^{L \times D}$. i and j are indices along the L (sequence length) dimension, and d is the index along the D (number of features) dimension.

$$
\begin{aligned}
&p_{ij} = \sum_d q_{id} v_{jd}; \quad p'_{ij} = \frac{p_{ij}}{\sqrt{D}} && \text{matmul, division} \\
&m_i = \max_j p'_{ij}; \quad e_{ij} = \exp(p'_{ij} - m_i); \quad s_i = \sum_j e_{ij} && \text{softmax (steps 1-3)} \\
&o_{id} = \sum_j e_{ij} v_{jd}; \quad o'_{id} = \frac{o_{id}}{s_i} && \text{matmul, softmax (step 4)}
\end{aligned}
\tag{12}
$$

Performers. Performers [8] is an alternative transformer architecture that uses non-softmax computation to approximate softmax. The performer operator takes three inputs Q, K, and V with a sequence length (L) and a number of features (D) dimension, similar to attention. Performer applies a randomly sampled projection matrix of shape $M \times D$ along with certain normalization on Q and K to produce $Q', K' \in \mathbb{R}^{L \times M}$. Then performer computes $A = Q'K'V$ and applies additional normalization on A without needing to compute a softmax. These normalization steps provide multiple reductions that Neptune's reduction fusion applies to.

Neptune fuses performer into two loop nests, applying reduction fusion four times. Two fusions have computation pattern g_1, and the other two have distinct computation patterns g_2 and g_3:

$$
\begin{aligned}
g_1(r_1, c_1, c_2) &= \alpha \exp(c_1 - r_1 - \beta c_2); & h_1(t, r, r') &= t \exp(r' - r) \\
g_2(r_1, c_1, c_2, c_3) &= \alpha c_3 \exp(c_1 - r_1 - \beta c_2); & h_2(t, r, r') &= t \exp(r' - r) \\
g_3(r_1, r_2, c_1, c_2, c_3) &= \alpha \exp(c_1 - r_1 - \beta c_2 c_3)/r_2; & h_3(t, r_1, r'_1, r_2, r'_2) &= t(r_2/r'_2) \exp(r_1 - r'_1)
\end{aligned}
$$

α and β are compile-time constants. The reduction function in all these cases is $f(x, y) = x + y$. g_1, g_2, and g_3 are all different from any softmax-based attention variants. The solution that Neptune finds is also given as h_1, h_2, and h_3. The last case g_3 is a fusion between one reduction and two producers, as the two reduction inputs r_1 and r_2 indicate, which Neptune is also capable of (§4.2, Eqns. 8 and 9).

Upstream optimizations can change the program that reaches Neptune fusion. One optimization is to delay the normalization on $A = Q'K'V$ to the end of the computation. This optimization can improve the performance and numerical stability of Neptune-fused kernel, as the normalization involves a division that is now delayed outside of the rolling loop and only applied once. When this optimization is applied, the fourth fusion (g_3) will have an identical compute pattern as the third one (g_2).

D Proofs of Rolling Update Correctness Theorems

We use these abbreviations for the following proofs

$$C[\phi_c(\boldsymbol{i}, \boldsymbol{j}')] =: C_{j'} \quad X_t[\phi_t(\boldsymbol{i})]^{\langle j \rangle} =: X_t^{\langle j \rangle} \quad X_r[\phi_r(\boldsymbol{i})]^{\langle j \rangle} =: X_r^{\langle j \rangle}.$$

essentially dropping the tensor access function and the map iteration vector $\boldsymbol{i}$, because all the proofs involve a single loop nest where $\boldsymbol{i}$ is fixed.

Proof of Lemma 4.3. We first show that the constructive solution of h, provided in Eq. 5, is a solution to condition Eq. 3. Since g_c^{-1} is the second-argument inverse of g, we have

$$g_c^{-1}(x, g(x, c)) = c$$

In Eq. 5, substitute $t = g(r, c)$ to get

$$h(g(r, c), r, r') = g(r', g_c^{-1}(r, g(r, c))) = g(r', c)$$

which is exactly Eq. 3. Next, given that h satisfies the conditions Eqns. 3 and 4, we prove that h satisfies Def. 4.2 which is an equality. The left-hand side of the equality is h applied to the reduce expression (rewritten using our abbreviations)

$$\mathcal{R}\left(f, 0 \preceq \boldsymbol{j}' \preceq \boldsymbol{j}_0, g\left(X_r^{\langle \boldsymbol{j}_f \rangle}, C_{j'}\right)\right) =: R_{\boldsymbol{j}_0}^{\langle \boldsymbol{j}_f \rangle}$$

and the right-hand side is the updated reduce expression

$$\mathcal{R}\left(f, 0 \preceq \boldsymbol{j}' \preceq \boldsymbol{j}_0, g\left(X_r^{\langle \boldsymbol{j}_t \rangle}, C_{j'}\right)\right) =: R_{\boldsymbol{j}_0}^{\langle \boldsymbol{j}_t \rangle}$$

The subscript $\boldsymbol{j}_0$ in $R_{\boldsymbol{j}_0}$ indicates the upper bound of the reduce range. Therefore, we are to prove

$$\forall \boldsymbol{j}_0, \boldsymbol{j}_f, \boldsymbol{j}_t \in \mathcal{D}^s, \boldsymbol{j}_f \preceq \boldsymbol{j}_t \preceq \boldsymbol{j}_0 \quad h\left(R_{\boldsymbol{j}_0}^{\langle \boldsymbol{j}_f \rangle}, X_r^{\langle \boldsymbol{j}_f \rangle}, X_r^{\langle \boldsymbol{j}_t \rangle}\right) = R_{\boldsymbol{j}_0}^{\langle \boldsymbol{j}_t \rangle}$$

which we prove by induction over $\boldsymbol{j}_0$. We introduce $\boldsymbol{j}$ as the induction variable to go from 0 to $\boldsymbol{j}_0$, while all tags are fixed, and maintain the inductive hypothesis

$$\forall 0 \preceq \boldsymbol{j} \preceq \boldsymbol{j}_0 \quad h\left(R_{\boldsymbol{j}}^{\langle \boldsymbol{j}_f \rangle}, X_r^{\langle \boldsymbol{j}_f \rangle}, X_r^{\langle \boldsymbol{j}_t \rangle}\right) = R_{\boldsymbol{j}}^{\langle \boldsymbol{j}_t \rangle} \tag{13}$$

- Base case: when $\boldsymbol{j} = 0$, $R_0^{\langle \boldsymbol{j}_f \rangle}$ and $R_0^{\langle \boldsymbol{j}_t \rangle}$ are each a single term, so we spell out these terms in the inductive hypothesis to get

  $$h\left(g\left(X_r^{\langle \boldsymbol{j}_f \rangle}, C_0\right), X_r^{\langle \boldsymbol{j}_f \rangle}, X_r^{\langle \boldsymbol{j}_t \rangle}\right) = g\left(X_r^{\langle \boldsymbol{j}_t \rangle}, C_0\right)$$

  which is true by applying Eq. 3.
- Inductive step: for any $\boldsymbol{j}$ where the inductive hypothesis Eq. 13 is true, we move on to $\text{next}(\boldsymbol{j})$ by adding one term to $R_{\boldsymbol{j}}^{\langle \boldsymbol{j}_f \rangle}$:

  $$R_{\text{next}(\boldsymbol{j})}^{\langle \boldsymbol{j}_f \rangle} = R_{\boldsymbol{j}}^{\langle \boldsymbol{j}_f \rangle} \textcircled{f}\, g\left(X_r^{\langle \boldsymbol{j}_f \rangle}, C_{\text{next}(\boldsymbol{j})}\right)$$

  Apply h to both sides, apply Eq. 4, then apply Eq. 3, and finally apply the inductive hypothesis

on the underlined term, to get

$$
\begin{aligned}
& h\left(R_{\text{next}(\boldsymbol{j})}^{\langle j_f \rangle}, X_r^{\langle j_f \rangle}, X_r^{\langle j_t \rangle}\right) = h\left(R_{\boldsymbol{j}}^{\langle j_f \rangle} \textcircled{f}\, g\left(X_r^{\langle j_f \rangle}, C_{\text{next}(\boldsymbol{j})}\right), X_r^{\langle j_f \rangle}, X_r^{\langle j_t \rangle}\right) \\
&= h\left(R_{\boldsymbol{j}}^{\langle j_f \rangle}, X_r^{\langle j_f \rangle}, X_r^{\langle j_t \rangle}\right) \textcircled{f}\, h\left(g\left(X_r^{\langle j_f \rangle}, C_{\text{next}(\boldsymbol{j})}\right), X_r^{\langle j_f \rangle}, X_r^{\langle j_t \rangle}\right) \\
&= \underline{h\left(R_{\boldsymbol{j}}^{\langle j_f \rangle}, X_r^{\langle j_f \rangle}, X_r^{\langle j_t \rangle}\right)} \textcircled{f}\, g\left(X_r^{\langle j_t \rangle}, C_{\text{next}(\boldsymbol{j})}\right) \\
&= R_{\boldsymbol{j}}^{\langle j_t \rangle} \textcircled{f}\, g\left(X_r^{\langle j_t \rangle}, C_{\text{next}(\boldsymbol{j})}\right) = R_{\text{next}(\boldsymbol{j})}^{\langle j_t \rangle}
\end{aligned}
$$

Therefore, the inductive hypothesis is true for next($\boldsymbol{j}$).

By induction, the theorem is true for all $\boldsymbol{j} \preceq \boldsymbol{j}_0$, and therefore true for $\boldsymbol{j}_0$. □

Proof of Theorem 4.1. The goal of this proof is to show that the result of rolling update transformed program matches that of the original program. We copy the explicit expression of X_t in the original program from Eq. 10, and apply our abbreviations:

$$
X_t^{\langle \boldsymbol{m} \rangle} = \mathcal{R}\left(f, 0 \preceq \boldsymbol{j}' \preceq \boldsymbol{m}, g\left(X_r^{\langle \boldsymbol{m} \rangle}, C_{\boldsymbol{j}'}\right)\right) \tag{14}
$$

On the other hand, the repaired program Eq. 7 is (with tag added for clarity)

$$
X_t^{\langle \boldsymbol{j} \rangle} = h\left(X_t^{\langle \text{prev}(\boldsymbol{j}) \rangle}, X_r^{\langle \text{prev}(\boldsymbol{j}) \rangle}, X_r^{\langle \boldsymbol{j} \rangle}\right) \textcircled{f}\, g\left(X_r^{\langle \boldsymbol{j} \rangle}, C_{\boldsymbol{j}}\right)
$$

and we assert it produces this value at iteration $\boldsymbol{j}$:

$$
X_t^{\langle \boldsymbol{j} \rangle} = \mathcal{R}\left(f, 0 \preceq \boldsymbol{j}' \preceq \boldsymbol{j}, g\left(X_r^{\langle \boldsymbol{j} \rangle}, C_{\boldsymbol{j}'}\right)\right) =: R_{\boldsymbol{j}}^{\langle \boldsymbol{j} \rangle} \tag{15}
$$

If this is true, then we have $X_t^{\langle \boldsymbol{m} \rangle} = X_t^{\langle \boldsymbol{m} \rangle}$ by substituting $\boldsymbol{m}$ for $\boldsymbol{j}$ in Eq. 15 and comparing to Eq. 14. We prove this Eq. 15 by induction on $\boldsymbol{j}$.

- Base case: when $\boldsymbol{j} = 0$, the recurrent X_0 has iterated once:

$$
X_t^{\langle 0 \rangle} = g\left(X_r^{\langle 0 \rangle}, C_0\right)
$$

  and the explicit $X_t^{\langle 0 \rangle}$ has a single term:

$$
R_0^{\langle 0 \rangle} = g\left(X_r^{\langle 0 \rangle}, C_0\right)
$$

  and they are equal, so the inductive hypothesis is true.
- Inductive step: for any $\boldsymbol{j}$ where the inductive hypothesis is true, the next iteration next($\boldsymbol{j}$) updates the recurrent X_t once. We write down the updated value, apply the inductive hypothesis to convert to explicit form, then use the tag-updating property of h to get

$$
\begin{aligned}
& X_t^{\langle \text{next}(\boldsymbol{j}) \rangle} \\
&= h\left(X_t^{\langle \boldsymbol{j} \rangle}, X_r^{\langle \boldsymbol{j} \rangle}, X_r^{\langle \text{next}(\boldsymbol{j}) \rangle}\right) \textcircled{f}\, g\left(X_r^{\langle \text{next}(\boldsymbol{j}) \rangle}, C_{\text{next}(\boldsymbol{j})}\right) \\
&= h\left(R_{\boldsymbol{j}}^{\langle \boldsymbol{j} \rangle}, X_r^{\langle \boldsymbol{j} \rangle}, X_r^{\langle \text{next}(\boldsymbol{j}) \rangle}\right) \textcircled{f}\, g\left(X_r^{\langle \text{next}(\boldsymbol{j}) \rangle}, C_{\text{next}(\boldsymbol{j})}\right) \\
&= R_{\boldsymbol{j}}^{\langle \text{next}(\boldsymbol{j}) \rangle} \textcircled{f}\, g\left(X_r^{\langle \text{next}(\boldsymbol{j}) \rangle}, C_{\text{next}(\boldsymbol{j})}\right) \\
&= R_{\text{next}(\boldsymbol{j})}^{\langle \text{next}(\boldsymbol{j}) \rangle}
\end{aligned}
$$

  Therefore, the inductive hypothesis is true for next($\boldsymbol{j}$).

By induction, Eq. 15 is true for all $\boldsymbol{j} \preceq \boldsymbol{m}$. Substituting $\boldsymbol{m}$ for $\boldsymbol{j}$, we get

$$X_t^{\langle \boldsymbol{m} \rangle} = \mathcal{R}\left(f, 0 \preceq \boldsymbol{j}' \preceq \boldsymbol{m}, g\left(X_r^{\langle \boldsymbol{m} \rangle}, C_{\boldsymbol{j}'}\right)\right)$$

which matches Eq. 14, and therefore proves the theorem. □

Proof of Theorem 4.4. The goal of this proof is to show that the result of split-k update transformed program matches that of the original program. The expected expression of X_t in the original program is the same as in Eq. 14:

$$X_t^{\langle \boldsymbol{m} \rangle} = \mathcal{R}\left(f, 0 \preceq \boldsymbol{j} \preceq \boldsymbol{m}, g\left(X_r^{\langle \boldsymbol{m} \rangle}, C_{\boldsymbol{j}}\right)\right)$$

Privatization splits this reduction into a local reduction, that produces $\boldsymbol{m}_0$ partial results, each result reduced from $\boldsymbol{m}_1$ terms. Iterators for $\boldsymbol{m}_0$ and $\boldsymbol{m}_1$ combine to an iterator for $\boldsymbol{m}$ similar to how loop tiling works:

$$\forall 0 \preceq \boldsymbol{j}_0 \preceq \boldsymbol{m}_0, 0 \preceq \boldsymbol{j}_1 \preceq \boldsymbol{m}_1 \quad \exists 0 \preceq \boldsymbol{j} \preceq \boldsymbol{m} \quad \boldsymbol{j}_0 \cdot \boldsymbol{m}_1 + \boldsymbol{j}_1 = \boldsymbol{j}$$

The result of the local reduction is defined by the local reduction that produces it:

$$X_{t,l}[\boldsymbol{j}_0] = X_{t,l}[\boldsymbol{j}_0] \textcircled{f}\, g\left(X_{r,l}[\boldsymbol{j}_0], C_{\boldsymbol{j}_0 \cdot \boldsymbol{m}_1 + \boldsymbol{j}_1}\right)$$

$\boldsymbol{j}_0$ shows up in the index of tensor $X_{t,l}$ because the local reduction produces $\boldsymbol{j}_0$ partial results. We convert this program into the following expression for $X_{t,l}$:

$$X_{t,l}[\boldsymbol{j}_0] = \mathcal{R}\left(f, 0 \preceq \boldsymbol{j}_1 \preceq \boldsymbol{m}_1, g\left(X_{r,l}[\boldsymbol{j}_0], C_{\boldsymbol{j}_0 \cdot \boldsymbol{m}_1 + \boldsymbol{j}_1}\right)\right)$$

The global reduction further combines these partial results into a single result over $\boldsymbol{j}_0$ iterations:

$$X_{t,g} = X_{t,g} \textcircled{f}\, h(X_{t,l}[\boldsymbol{j}_0], X_{r,l}[\boldsymbol{j}_0], X_{r,g})$$

We substitute the expression of $X_{t,l}[\boldsymbol{j}_0]$ into this program to get

$$X_{t,g} = X_{t,g} \textcircled{f}\, h(\mathcal{R}\left(f, 0 \preceq \boldsymbol{j}_1 \preceq \boldsymbol{m}_1, g\left(X_{r,l}[\boldsymbol{j}_0], C_{\boldsymbol{j}_0 \cdot \boldsymbol{m}_1 + \boldsymbol{j}_1}\right)\right), X_{r,l}[\boldsymbol{j}_0], X_{r,g})$$

Using the tag-updating property of h, the expression that we expanded from $X_{t,l}[\boldsymbol{j}_0]$ drops all $X_{r,l}[\boldsymbol{j}_0]$ and replaces them with $X_{r,g}$:

$$X_{t,g} = X_{t,g} \textcircled{f}\, \mathcal{R}\left(f, 0 \preceq \boldsymbol{j}_1 \preceq \boldsymbol{m}_1, g\left(X_{r,g}, C_{\boldsymbol{j}_0 \cdot \boldsymbol{m}_1 + \boldsymbol{j}_1}\right)\right)$$

which has the following explicit expression:

$$\begin{aligned} X_{t,g} &= \mathcal{R}\left(f, 0 \preceq \boldsymbol{j}_0 \preceq \boldsymbol{m}_0, \mathcal{R}\left(f, 0 \preceq \boldsymbol{j}_1 \preceq \boldsymbol{m}_1, g\left(X_{r,g}, C_{\boldsymbol{j}_0 \cdot \boldsymbol{m}_1 + \boldsymbol{j}_1}\right)\right)\right) \\ &= \mathcal{R}\left(f, 0 \preceq \boldsymbol{j} \preceq \boldsymbol{m}, g\left(X_{r,g}, C_{\boldsymbol{j}}\right)\right) \end{aligned}$$

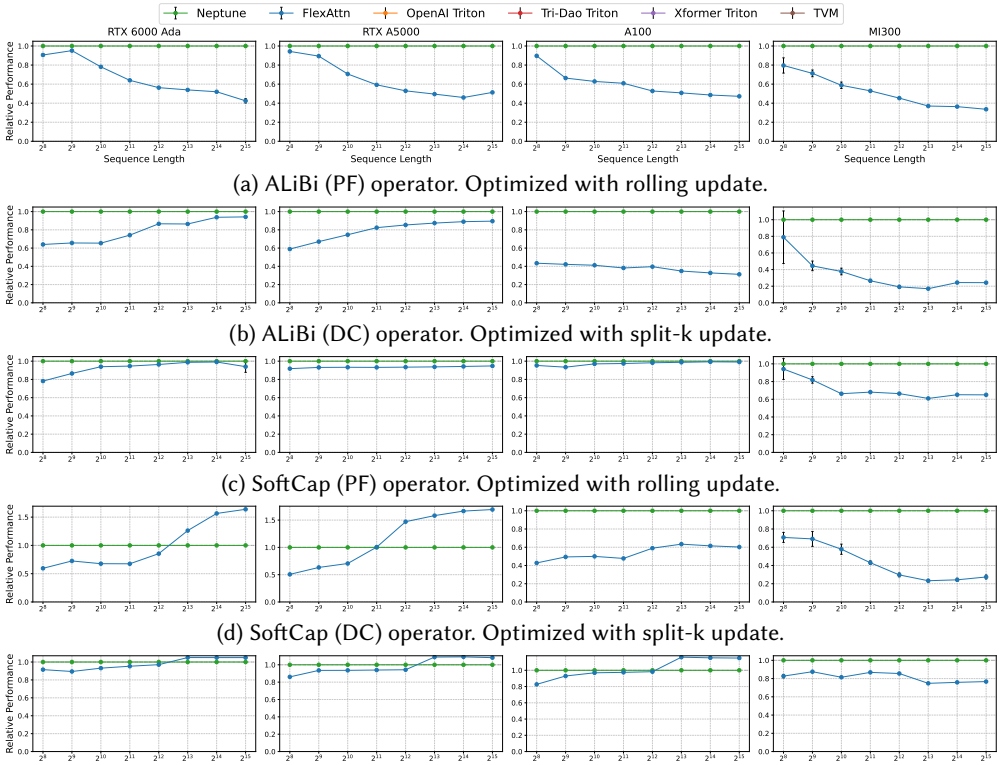
which matches the expected expression of $X_{t,g}$ in the original program. □

E Additional Evaluation

E.1 Compilation Statistics

We provide some statistics Neptune's compilation pipeline when generating kernels we have evaluated in Section 8. For these evaluated operators, Neptune with 128 autotuning iterations takes 1.5 to 10 minutes to run on each setup. Triton-based tile optimization takes 50% to 90% of the time (depending on the input shape). Neptune's template-guided optimization (rolling/split-k update) takes 5% to 10% of the time (does not depend on input shape), and the rest is TVM autotuning search algorithm.

E.2 Operator Evaluation



(a) ALiBi (PF) operator. Optimized with rolling update.

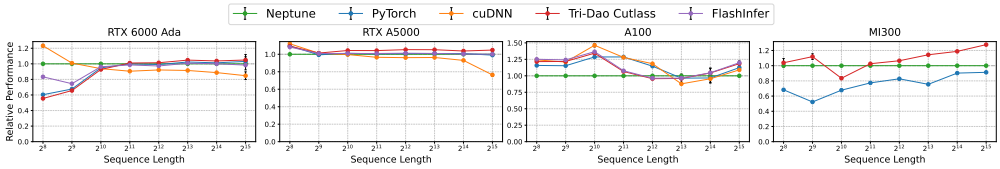
(b) ALiBi (DC) operator. Optimized with split-k update.

(c) SoftCap (PF) operator. Optimized with rolling update.

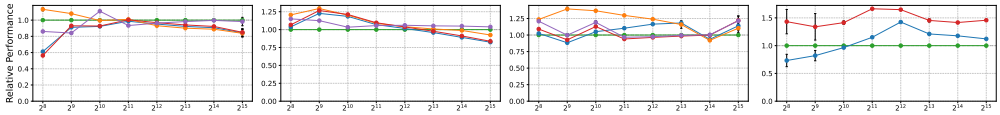
(d) SoftCap (DC) operator. Optimized with split-k update.

(e) Window (PF) operator. Optimized with rolling update.

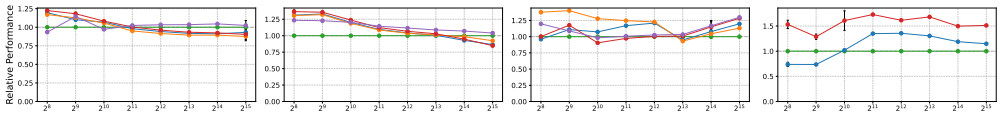
Fig. 16. Performance of Neptune vs. other tensor compilers on the five operators not shown in Figure 8.



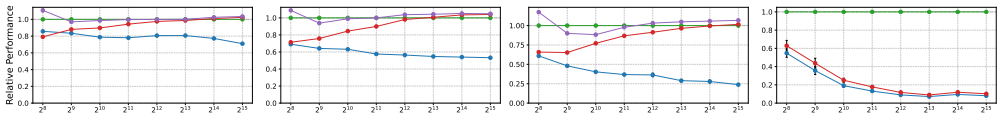
(a) Global (PF) operator. Optimized with rolling update.



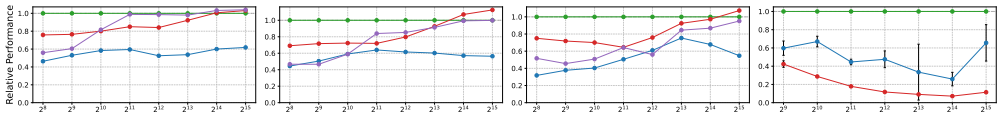
(b) Causal (PF) operator. Optimized with rolling update.



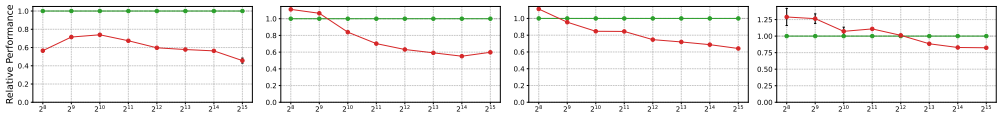
(c) GQA (PF) operator. Optimized with rolling update.



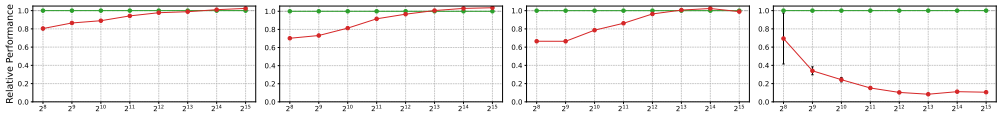
(d) Causal (DC) operator. Optimized with split-k update.



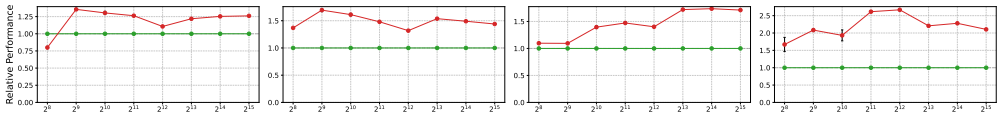
(e) GQA (DC) operator. Optimized with split-k update.



(f) ALiBi (PF) operator. Optimized with rolling update.



(g) ALiBi (DC) operator. Optimized with split-k update.



(h) Window (PF) operator. Optimized with rolling update.

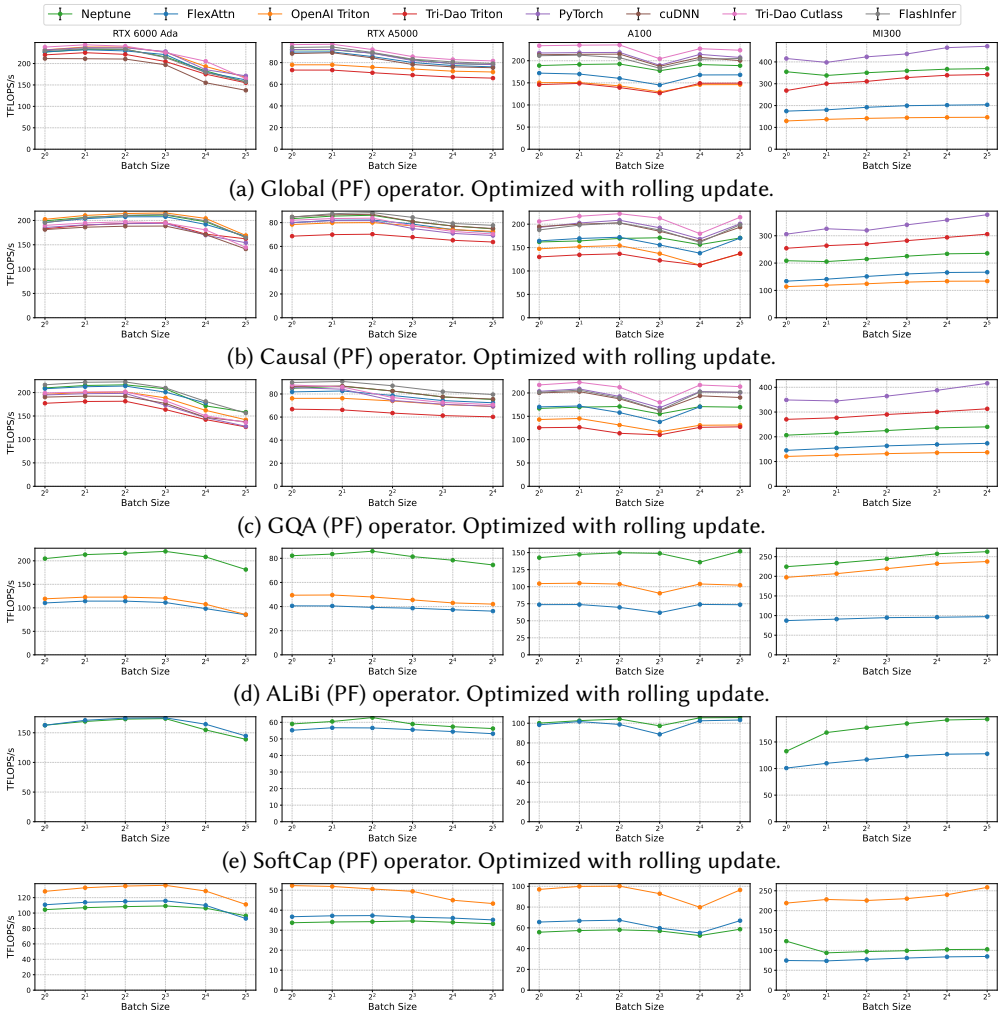
Fig. 17. Performance of Neptune vs. tensor libraries on 8 operators for which library baselines exist (we were unable to find the library implementations of SoftCap PF/DC).

E.3 Scalability Test

Figure 18 extends the scalability test of Figure 9 to all the 10 operators, 4 GPUs and all implementations used in our Evaluation section. We still use a sequence length of 8192 and batch sizes up to 32.

Figures 18a to 18f shows that the throughput of all prefill kernels decrease as input batch size increases. By closely inspecting profiling results, we find that larger workloads are more likely to trigger GPUs' clock throttling. Figure 19 shows a profile in the Nvidia Nsight System profiler, with a kernel (`flash_fwd_kernel`) that runs for 281 milliseconds. The GPU compute clock ("GPC Clock Frequency") starts to throttle within 50 milliseconds after the kernel starts.

In Figures 18g to 18j, the decoding kernels exhibit the opposite trend: throughput increases with batch size. Decoding kernels are much less compute-bound than prefill kernels to trigger clock throttling, and benefit from increasing parallelism at larger batch sizes.



(a) Global (PF) operator. Optimized with rolling update.

(b) Causal (PF) operator. Optimized with rolling update.

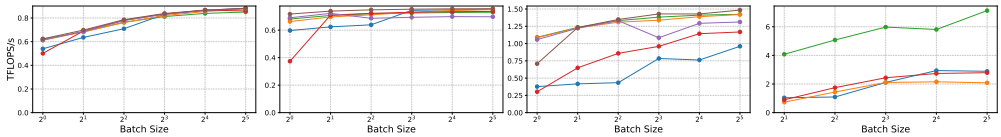
(c) GQA (PF) operator. Optimized with rolling update.

(d) ALiBi (PF) operator. Optimized with rolling update.

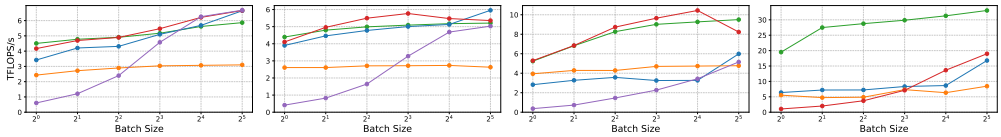
(e) SoftCap (PF) operator. Optimized with rolling update.

(f) Window (PF) operator. Optimized with rolling update.

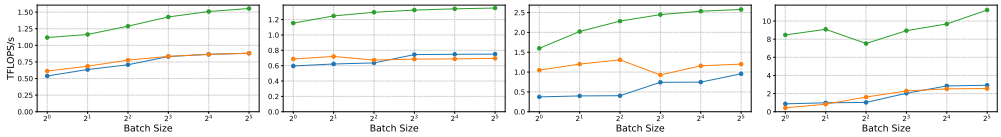
Fig. 18. Throughput of Neptune kernels and multiple baselines over increasing batch size, for all 10 operators on all 4 GPUs. The y-axis shows throughput in TFLOPS/sec, and the x-axis shows batch size.



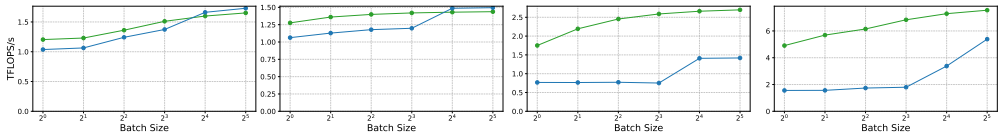
(g) Causal (DC) operator. Optimized with split-k update.



(h) GQA (DC) operator. Optimized with split-k update.



(i) ALiBi (DC) operator. Optimized with split-k update.



(j) SoftCap (DC) operator. Optimized with split-k update.

Fig. 18. (Continued) Throughput of Neptune kernels and multiple base- lines over increasing batch size, for all 10 operators on all 4 GPUs. The y-axis shows throughput in TFLOPS/sec, and the x-axis shows batch size.

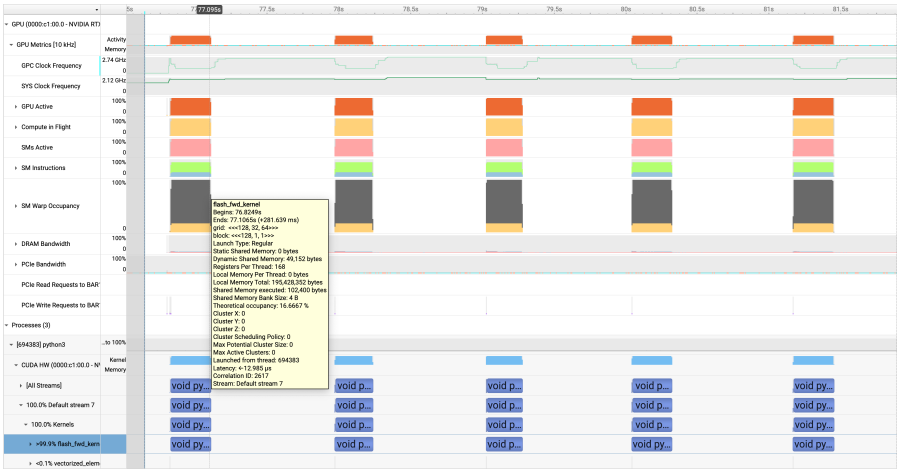
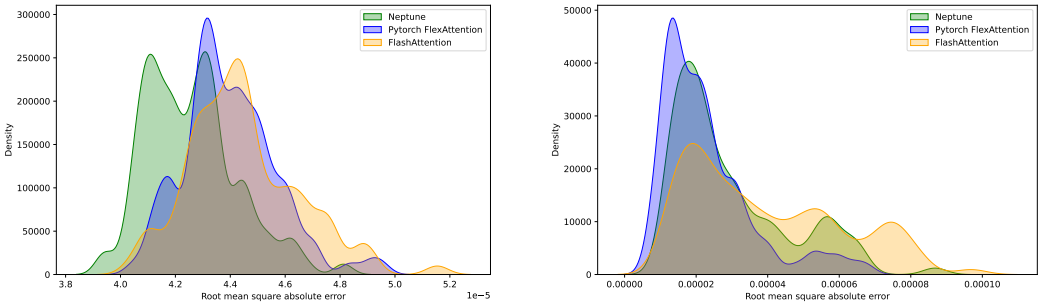


Fig. 19. Timeline view of a profile in the Nvidia Nsight System profiler. The bottom rows show when the kernels are executing, and the “GPC Clock Frequency” row near the top shows the frequency of the GPU graphic clock over time.



(a) Prefill (b) Decoding

Fig. 20. Per-sample GQA RMSE for (a) prefill and (b) decoding phases.

F Numerical Stability of Neptune Kernels

Table 5. Numerical Error Summary (Absolute Error)

Variant	Neptune			Tri-Dao Attention			Flex Attention		
	RMS	90th %	99th %	RMS	90th %	99th %	RMS	90th %	99th %
PF Global	4.2e-05	1.5e-05	1.2e-04	4.2e-05	1.5e-05	1.2e-04	4.4e-05	1.5e-05	1.2e-04
PF Causal	4.1e-05	1.9e-05	1.2e-04	4.2e-05	2.5e-05	1.2e-04	4.2e-05	2.3e-05	1.2e-04
PF GQA	4.3e-05	2.3e-05	1.2e-04	4.5e-05	3.1e-05	1.2e-04	4.4e-05	2.7e-05	1.2e-04
PF ALiBi	4.3e-05	1.5e-05	1.2e-04	5.4e-05	3.1e-05	2.4e-04	4.3e-05	1.5e-05	1.2e-04
PF Softcap	2.4e-05	7.6e-06	3.1e-05	–	–	–	2.4e-05	7.6e-06	3.1e-05
PF Windowed	2.4e-05	7.6e-06	3.1e-05	2.5e-05	7.6e-06	6.1e-05	2.4e-05	7.6e-06	3.1e-05
DC Causal	3.9e-05	1.5e-05	1.2e-04	4.6e-05	1.5e-05	1.2e-04	3.0e-05	1.5e-05	1.2e-04
DC GQA	3.4e-05	1.5e-05	1.2e-04	4.5e-05	1.5e-05	1.2e-04	2.6e-05	7.6e-06	1.2e-04
DC ALiBi	4.3e-05	1.5e-05	1.2e-04	4.8e-05	1.5e-05	2.4e-04	4.1e-05	1.5e-05	1.2e-04
DC Softcap	1.4e-05	1.9e-06	3.1e-05	–	–	–	1.3e-05	9.5e-07	3.1e-05

Optimizations that rewrite or re-associate floating-point operations may alter the numerical properties of a kernel. We evaluate the numerical behavior of Neptune, Tri-Dao, and FlexAttention kernels on a variety of attention operators, and find Neptune to provide competitive numerical behavior. Using a set of common inputs, we evaluate kernel outputs against an FP64 unfused attention kernel baseline. Each kernel uses the same schedule and configuration as the performance evaluation. We sample realistic attention inputs by running a pretrained Qwen2.5 model on 100 sentences with sequence length 2048 from the WikiText dataset and recording the input tensors.

Table 5 presents the numerical error of all three frameworks, represented by the root mean square error (RMS) and the magnitudes of the 90th and 99th percentile errors. We also evaluate the distribution of per-sample (per-sentence) RMSE, with results for GQA prefill and decoding show in Figure 20. For all prefill benchmarks, Neptune kernels have lower error than the other two frameworks. For the decode benchmarks, Neptune has lower error than Tri-Dao Attention (when present) and comparable error to FlexAttention.

Received 2025-11-14; accepted 2026-04-03